Lab 4: dane na urządzeniu (SwiftData)

Zaawansowane programowanie na platformę iOS

Na tych zajęciach aplikacja zaczyna zapamiętywać dane na telefonie. Dochodzą trzy rzeczy:

- lista filmów do obejrzenia, nowa zakładka Saved,
- historia wyszukiwania na ekranie Search,
- ostatnie odpowiedzi TMDB, pokazywane bez internetu.

Wszystko zapisuje SwiftData, baza danych wbudowana w iOS. Dane zostają po zamknięciu aplikacji.

Cały kod piszą Państwo sami. Ta instrukcja mówi, co ma powstać i gdzie. Nazwy plików, typów i funkcji proszę zachować, bo według nich sprawdzam prace.

Zanim Państwo zaczną

1. Lab 3 musi być skończony. Aplikacja pobiera filmy i szczegóły z TMDB.

Pliki

Nowe:

```
MyNextMovie/Storage/StoredModels.swift   modele SwiftData
MyNextMovie/Storage/Storage.swift        zapis i odczyt z bazy
MyNextMovie/Views/SavedMoviesView.swift  zakładka Saved i przycisk zapisu
MyNextMovieTests/StorageTests.swift      testy
```

Do zmiany:

```
MyNextMovie/MyNextMovieApp.swift         baza danych, zakładka Saved,
zapytania z cache
MyNextMovie/Services/TMDB.swift          zapytania z cache
MyNextMovie/Views/MovieDetailView.swift  przycisk zapisu na pasku
MyNextMovie/Views/MovieListView.swift    MovieGrid dostępny poza plikiem
MyNextMovie/Views/SearchView.swift       historia wyszukiwania
```

Jak działa SwiftData

Model to klasa z `@Model`. Każde jej pole jest kolumną w bazie:

```
@Model
final class SavedMovie {
    @Attribute(.unique) var movieId: Int
    var title: String
    var savedAt: Date
    // ...
}
```

`@Attribute(.unique)` pilnuje, żeby wartość była w bazie najwyżej raz. To klasa, a nie struktura, bo `@Model` tego wymaga. Klasa potrzebuje `init`, który ustawia wszystkie pola. Bazę dla całej aplikacji tworzy modyfikator na `WindowGroup`:

```
.modelContainer(for: storedModelTypes)
```

Z bazy korzysta się na dwa sposoby:

1. W widoku `@Query` pobiera obiekty i sam odświeża widok, gdy coś się zmieni:

   ```
   @Query(sort: \SavedMovie.savedAt, order: .reverse) private var
   savedMovies: [SavedMovie]
   ```

2. W zwykłej funkcji przez `ModelContext`:

   ```
   context.insert(obiekt)          // dodaj
   context.delete(obiekt)          // usuń
   try? context.fetch(descriptor)  // pobierz
   ```

   `FetchDescriptor<SavedMovie>` mówi, co pobrać. Ma `predicate`, czyli warunek jak `where` w SQL, `sortBy` i `fetchLimit`. Warunek zapisuje się makrem: `#Predicate { savedMovie in savedMovie.movieId == movieId }`.

Widok bierze kontekst z `@Environment(\.modelContext) private var modelContext`.

Zmiany zapisują się same, z małym opóźnieniem. Nie trzeba wołać `save()`. Obiekt zmienia się w bazie przez zwykłe przypisanie do pola.
Dwie pułapki SwiftData:

- `context.delete(model:)` usuwa tylko obiekty już zapisane na dysku. Proszę pobrać obiekty przez `fetch` i usuwać je po jednym w pętli.
- `fetchOffset` też pomija obiekty jeszcze niezapisane. Proszę pobrać całą posortowaną listę i liczyć w pętli.

Funkcje w `Storage.swift` mają dostawać `context` jako parametr, a aktualną datę jako parametr `now`. Aplikacja podaje swój kontekst i `Date.now`, testy bazę w pamięci i stałą datę. Dzięki temu testy wiedzą, która data jest nowsza.

Zadanie 1: lista do obejrzenia

`Storage/StoredModels.swift`

- Model `SavedMovie`: wszystkie pola `Movie` (`movieId` zamiast `id`, unikalne) i `savedAt: Date`. `init(movie: Movie, savedAt: Date)` przepisuje pola z filmu. Zapisany film ma wszystkie dane, więc zakładka Saved działa bez sieci.
- Stała `storedModelTypes: [any PersistentModel.Type]`: lista wszystkich modeli, na razie `[SavedMovie.self]`. W zadaniach 2 i 3 dochodzą kolejne.

`Storage/Storage.swift`

Funkcja	Co robi
`fetchSavedMovies(_ context: ModelContext, movieId: Int) -> [SavedMovie]`	zapisane kopie filmu, pusta tablica albo jedna
`isMovieSaved(_ context: ModelContext, movieId: Int) -> Bool`	czy film jest zapisany
`toggleSavedMovie(_ context: ModelContext, _ movie: Movie, now: Date)`	zapisuje film, a gdy jest już zapisany, usuwa go
`movieFromSaved(_ savedMovie: SavedMovie) -> Movie`	zapisana kopia z powrotem jako `Movie`
`moviesFromSaved(_ savedMovies: [SavedMovie]) -> [Movie]`	to samo dla tablicy

Widoki i nawigacja używają `Movie`, stąd `movieFromSaved`.

`MyNextMovieApp.swift`

- `.modelContainer(for: storedModelTypes)` na `WindowGroup`.
- Nowa zakładka „Saved” z ikoną `bookmark`, między Movies i Search, z `SavedMoviesView`.

Nazwy ikon proszę dodać jako stałe w `Views/Components/Layout.swift`, obok `symbolNameMovies`.

`Views/SavedMoviesView.swift`

`SavedMoviesView`:

- `@Query` zapisanych filmów, najnowszy pierwszy.
- Pusta lista: `ContentUnavailableView` z tekstem „No saved movies” i podpowiedzią, że film zapisuje się zakładką na ekranie szczegółów.
- Inaczej siatka `MovieGrid` z lab 1, tytuł „Watch Later”. Żeby `MovieGrid` był dostępny w tym pliku, w `MovieListView.swift` proszę usunąć przed nim `private`.
- Własny `NavigationStack` i `.navigationDestination(for: Movie.self)`, jak w `MovieListView`. Do szczegółów przekazuje `loadDetails`, który dostaje w `init`.

`SaveMovieButton(movie: Movie)`: przycisk z ikoną `bookmark`, a dla zapisanego filmu `bookmark.fill`. Kliknięcie woła `toggleSavedMovie`. Widok potrzebuje `@Query` tylko dla swojego filmu. Takie zapytanie tworzy się w `init`:

```
@Query private var savedCopies: [SavedMovie]

init(movie: Movie) {
    self.movie = movie
    let movieId = movie.id
    _savedCopies = Query(filter: #Predicate { savedMovie in savedMovie.movieId
== movieId })
}
```

W `MovieDetailView` przycisk trafia na pasek: `.toolbar { SaveMovieButton(movie: movie) }`.

Podgląd widoku, który używa `@Query` albo `modelContext`, potrzebuje bazy. Bez niej podgląd się wywraca. Proszę dodać do każdego takiego `#Preview` bazę w pamięci:

```
.modelContainer(for: storedModelTypes, inMemory: true)
```

Dotyczy to też podglądów `MovieListView`, `SearchView` i `MovieDetailView`, bo prowadzą do przycisku zapisu.

Testy w `MyNextMovieTests/StorageTests.swift`

Każdy test tworzy własną, pustą bazę w pamięci. Pomocnicza funkcja:

```
@MainActor
func makeTestContainer() throws -> ModelContainer {
    let configuration = ModelConfiguration(isStoredInMemoryOnly: true)
    return try ModelContainer(for: Schema(storedModelTypes), configurations:
[configuration])
}
```

W teście: `let container = try makeTestContainer()` i `let context = container.mainContext`. Kontener musi żyć do końca testu, dlatego jest w zmiennej. Struktura `SavedMovieTests`:

Test	Sprawdza
`toggleSavesThenRemoves`	pierwsze `toggleSavedMovie` zapisuje, drugie usuwa
`savedCopyKeepsEveryField`	`moviesFromSaved` z bazy daje ten sam `Movie`, który zapisano
`savingTwiceKeepsOneCopy`	po zapisie, usunięciu i ponownym zapisie w bazie jest jeden film

Proszę sprawdzić w aplikacji:

1. Zakładka na pasku szczegółów zapisuje film. Ikona się wypełnia.
2. Film jest na zakładce Saved, najnowszy pierwszy.
3. Drugie kliknięcie usuwa film z listy.
4. Zapisane filmy zostają po ponownym uruchomieniu aplikacji. Przed zatrzymaniem proszę odczekać kilka sekund, bo SwiftData zapisuje zmiany z opóźnieniem.

Zadanie 2: historia wyszukiwania

`Storage/StoredModels.swift`

Model `SearchHistoryEntry`: `query: String` (unikalne) i `searchedAt: Date`. Do `storedModelTypes` proszę dopisać `SearchHistoryEntry.self`.

`Storage/Storage.swift`

Stała `maxSearchHistoryCount = 10`.

Funkcja	Co robi
`recordSearch(_ context: ModelContext, query: String, now: Date)`	zapamiętuje wyszukiwanie
`recentSearches(_ context: ModelContext) -> [SearchHistoryEntry]`	najnowsze pierwsze, najwyżej `maxSearchHistoryCount`
`clearSearchHistory(_ context: ModelContext)`	usuwa całą historię

Zasady `recordSearch`:

- Obcina spacje wokół tekstu. Pustego tekstu nie zapisuje.
- Tekst, który już jest w historii, nie tworzy nowego wpisu. Dostaje nową datę i przez to wraca na górę.
- Po zapisie zostawia `maxSearchHistoryCount` najnowszych wpisów, starsze usuwa. Proszę to wydzielić do prywatnej funkcji `removeOldSearches`.

`Views/SearchView.swift`

- Wyszukiwanie zapisuje się, gdy użytkownik naciśnie Search (Return) na klawiaturze: modyfikator `.onSubmit(of: .search)` obok `.searchable`. Zapis po każdej literze dałby w historii „m", „ma", „mat".
- Zamiast `ContentUnavailableView` w stanie `.idle` nowy widok `SearchStartView`. Z pustą historią pokazuje ten sam komunikat co dotąd. Inaczej `List` z sekcją „Recent Searches". W nagłówku sekcji przycisk „Clear", który czyści historię.
- Wiersz historii to przycisk z ikoną `clock.arrow.circlepath` i tekstem. Kliknięcie wpisuje tekst do pola wyszukiwania (`viewModel.query`) i zapisuje go ponownie w historii. `SearchStartView` nie ma dostępu do view modelu, więc dostaje od `SearchView` funkcję `selectQuery: (String) -> Void`, tak jak `GenreFilter` dostaje `toggleGenre`.

Testy, struktura `SearchHistoryTests`

Test	Sprawdza
`newestSearchComesFirst`	„matrix" w minucie 1, „alien" w minucie 2: kolejność „alien", „matrix"
`repeatedSearchMovesToTop`	„matrix", „alien", znów „matrix": kolejność „matrix", „alien"
`emptyAndSpacesAreNotRecorded`	`""`, `"  "` i `"  dune  "` zostawiają tylko „dune"
`keepsOnlyRecentSearches`	po 13 wyszukiwaniach w bazie jest 10 wpisów, pierwszy to ostatnie
`clearRemovesEverything`	po `clearSearchHistory` historia jest pusta

Daty w testach: `Date(timeIntervalSince1970: 0)` i kolejne minuty przez `addingTimeInterval(60 * minuty)`.

Proszę sprawdzić w aplikacji:

1. „matrix” i Return, potem wyczyszczone pole: „matrix” jest w Recent Searches.
2. Kliknięcie we wpis wyszukuje go ponownie.
3. „Clear” czyści historię.

Zadanie 3: praca bez internetu

Aplikacja zapamiętuje ostatnią odpowiedź TMDB dla listy popularnych i dla szczegółów każdego otwartego filmu. Gdy zapytanie się nie uda, pokazuje zapamiętaną.

`Storage/StoredModels.swift`

Model `CachedResponse`: `key: String` (unikalne), `data: Data` (surowy JSON z TMDB) i `fetchedAt: Date`. Do `storedModelTypes` proszę dopisać `CachedResponse.self`.

`Storage/Storage.swift`

Funkcja	Co robi
`storeCachedResponse(_ context: ModelContext, key: String, data: Data, now: Date)`	zapisuje JSON pod kluczem. Jeden klucz to jedna odpowiedź, nowa zastępuje starą
`cachedResponseData(_ context: ModelContext, key: String) -> Data`	zapisany JSON, pusty `Data()` gdy nie ma
`fetchWithCacheFallback(_ context: ModelContext, key: String, fetch: () async throws -> Data) async throws -> Data`	najpierw `fetch()`, wynik zapisuje i zwraca. Gdy `fetch()` rzuci błąd, zwraca zapisany JSON. Gdy go nie ma, rzuca ten sam błąd dalej

W bloku `catch` złapany błąd nazywa się `error`. Rzuca się go dalej przez `throw error`.

`Services/TMDB.swift`

- `loadPopularMovies(cache: ModelContext) async throws -> [Movie]`: jak `fetchPopularMovies`, ale dane bierze z `fetchWithCacheFallback` z kluczem `"popular"`.
- `loadTMDBMovieDetails(movieId: Int, cache: ModelContext) async throws -> MovieDetails`: jak `fetchMovieDetails`, z kluczem `"movie/"` plus id, np. `"movie/603"`.

Cache trzyma surowy JSON, a nie gotowe filmy. Dzięki temu po zmianie struktur wystarczy zdekodować go jeszcze raz.

Wyszukiwanie zostaje bez cache. Bez internetu i tak nie znajdzie nowych filmów.

`MyNextMovieApp.swift`

Nowe funkcje potrzebują kontekstu, a view modele dostają `MovieLoader` i `MovieDetailsLoader`, czyli funkcje bez kontekstu. Łączy je `RootView`:

- dostaje `@Environment(\.modelContext) private var modelContext`,
- ma dwie prywatne funkcje, np. `loadPopularMoviesWithCache()` i `loadDetailsWithCache(movieId:)`, o typach `MovieLoader` i `MovieDetailsLoader`. Każda woła wersję z cache i podaje jej `modelContext`,
- przekazuje je do ekranów zamiast `fetchPopularMovies` i `fetchMovieDetails`, na wszystkich trzech zakładkach.

View modele i widoki zostają bez zmian. To ten sam pomysł co w lab 3: zmienia się tylko funkcja podana z zewnątrz.

Testy, struktura `ResponseCacheTests`

Test	**Sprawdza**
`missingKeyGivesEmptyData`	brak klucza daje pusty `Data`
`storingAgainReplacesData`	drugi zapis pod tym samym kluczem zastępuje dane, w bazie jest jeden obiekt
`freshAnswerIsReturnedAndCached`	wynik `fetch` jest zwrócony i zapisany
`cachedAnswerIsReturnedWhenOffline`	`fetch` rzuca `NoConnection()`, wynik to zapisany JSON
`errorIsRethrownWhenNothingCached`	`fetch` rzuca `NoConnection()`, nic nie ma w cache, test oczekuje `NoConnection`

Liczbę obiektów w bazie zwraca `try context.fetchCount(FetchDescriptor<CachedResponse>())`. Test funkcji `async`, która ma rzucić błąd: `await #expect(throws: NoConnection.self) { try await ... }`. Proszę sprawdzić w aplikacji:

1. Z internetem: otworzyć szczegóły dwóch filmów.
2. Wyłączyć Wi-Fi w Macu. Zatrzymać i uruchomić aplikację ponownie (⌘R).
3. Lista popularnych filmów jest na ekranie. Szczegóły tych dwóch filmów też.
4. Szczegóły innego filmu pokazują błąd, bo tego filmu nie ma w cache.
5. Plakaty mogą się nie pokazać. `AsyncImage` nie zapisuje obrazków na stałe.

Dokumentacja

- SwiftData: https://developer.apple.com/documentation/swiftdata
- Zapisywanie danych między uruchomieniami: https://developer.apple.com/documentation/swiftdata/preserving-your-apps-model-data-across-launches
- `Query`: https://developer.apple.com/documentation/swiftdata/query
- `ModelContext`: https://developer.apple.com/documentation/swiftdata/modelcontext
- `FetchDescriptor`: https://developer.apple.com/documentation/swiftdata/fetchdescriptor
- `#Predicate`: https://developer.apple.com/documentation/foundation/predicate
- `onSubmit(of:_:)`: https://developer.apple.com/documentation/swiftui/view/onsubmit(of:_:)
- `toolbar`: https://developer.apple.com/documentation/swiftui/view/toolbar(content:)